DESIGN DOCUMENT · IMPLEMENTED

Accounts, roles and an audit trail

Optional multi-user mode for Cubeherd: personal accounts, custom roles that can be granted globally, per node or per server, an audit log, and sign-in with Google or any OpenID Connect provider.

Decisions taken: scopes down to a single server · custom roles · audit log is mandatory (so SQLite from the start) · SSO optional, Google plus generic OIDC. Builds on the node model described in `multi-node-design.pdf`.

1. Summary

Today Cubeherd has one shared password. A session cookie says "logged in" and nothing more: there is no notion of who did something, and everyone who can log in can delete every server, read every config file and see every token.

This proposal adds an **opt-in accounts mode**. Without it nothing changes: one password, JSON files, no database. With it:

- **Users** sign in with a password or through SSO. Sessions are server-side and revocable.
- **Permissions** are fine grained (one per kind of action). A **role** is a named set of permissions; built-in roles are fixed, custom roles are freely composed.
- A **grant** gives a user (or group) a role on a **scope**: everything, one node, or one server. Grants are additive and inherit downwards.
- Every state-changing action and every login event lands in an **audit log** that cannot be edited from the UI.
- Users, roles, grants, sessions and the audit log live in **SQLite** (`cubeherd.db`). Server definitions stay in their folders, so a server still works without any Cubeherd process.

> **Core idea**
> One question answers every request: *may this user perform permission P on scope S?* A single function, `authz.Can(user, perm, scope)`, backs both the middleware that guards routes and the UI that hides buttons. A test fails the build when a registered route has no permission assigned.

What stays the same

- Single-user mode is the default and stays byte-for-byte compatible (no SQLite file is created).
- Agents keep their one bearer token. They do not know about users; the dashboard enforces rights (see section 7).
- Servers, backups and the S3 setup are untouched.

What changes

- A new **Access** area in the settings: Users, Roles, Grants, Sign-in (SSO), Audit log.
- Overview, sidebar, live streams and search only show what the signed-in user may see.
- A dependency on a pure-Go SQLite driver (see section 3).

2. Permission model

Scope hierarchy

```
global                      (the whole dashboard)
 └─ node   "Berlin A"       (one host)
     └─ server  "survival" (one Minecraft server)
```

A grant on a node applies to every server on it, including servers created later. A grant on **global** applies everywhere. Permissions are **additive only**: there are no deny rules, so the effective rights of a user are the union of all their grants, which keeps the model explainable ("why can she do this?" always has a visible answer in the UI).

> **Sensitive permissions**
> `files.read` can expose secrets (the RCON password sits in `server.properties`), `server.command` is operator-level control of the game, and `nodes.manage` reveals agent tokens. The role editor marks these with a warning so nobody hands them out by accident. Permissions whose lowest scope is global can never be granted on a node or server scope.

Permission catalogue

PERMISSION	ALLOWS	LOWEST SCOPE
`server.view`	See the server, status, players, logs, metrics	server
`server.power`	Start, stop, restart, pause, resume	server
`server.console`	Read the live console	server
`server.command`	Send console / RCON commands (effectively server operator)	server
`server.players`	Whitelist, ops, bans	server
`server.settings`	Change version, memory, ports, game settings	server
`server.update`	Update or switch the Minecraft version	server
`server.create`	Create or import a server	node
`server.delete`	Uninstall a server and its data	server
`files.read`, `files.write`	Browse and download / edit, upload, delete files	server
`world.export`, `world.import`	Download / replace the world	server
`addons.manage`	Install, remove, upload mods and plugins	server
`backups.view` / `.create` / `.restore` / `.delete`	List and download / create and verify / restore / delete backups	server
`nodes.view`	See the load and health of a node (CPU, RAM, disk, agent); without it only the permitted servers show	node
`nodes.manage`	Add, edit, remove nodes (tokens!)	global
`settings.manage`	Global settings: S3, CurseForge key, time zone, language	global
`users.manage`, `roles.manage`, `sso.manage`	Manage accounts, groups, roles and grants / configure identity providers	global

`audit.view` / `audit.export`	Read / export the audit log (entries of the granted scope)	server

3. Roles, grants and storage

Built-in roles

Built-in roles are read-only but can be cloned into a custom role.

PERMISSION GROUP	OWNER	ADMIN	OPERATOR	MODERATOR	VIEWER
View server, console, backups list	✓	✓	✓	✓	✓
Power, players	✓	✓	✓	✓	–
Commands	✓	✓	✓	✓	–
Settings, update, add-ons, backups create/restore	✓	✓	✓	–	–
Files, world import/export	✓	✓	✓	–	–
Create / delete servers	✓	✓	–	–	–
Nodes, global settings	✓	✓	–	–	–
Users, roles, SSO, audit log	✓	–	–	–	–

Data model

```
users        id, name, email, password_hash?, disabled, created_at, last_login_at
identities   user_id, issuer, subject, email            -- SSO links
groups       id, name
group_members group_id, user_id
roles        id, name, builtin, description
role_perms   role_id, permission
grants       id, principal_type(user|group), principal_id, role_id,
             scope_type(global|node|server), node_id?, server_id?,
             created_by, created_at, expires_at?
sessions     id_hash, user_id, created_at, last_seen_at, ip, user_agent, expires_at
audit        id, ts, actor_id, actor_name, ip, action, scope, target, outcome,
             detail(json), prev_hash, hash
```

Why SQLite, and which one

- The audit log needs append-only writes, filtering by user, action and time, and retention. Users and grants then belong in the same transactional store (a grant change and its audit row commit together).
- Driver: `modernc.org/sqlite` (pure Go). The build stays `CGO_ENABLED=0` and the image stays a static binary. Measured: the stripped binary grows from 11.1 MB to 15.5 MB, and a dashboard with accounts idles around 40 MB of memory; the compose file for a dashboard now allows 96 MB. The file is opened in WAL mode with a busy timeout.
- Server data is **not** moved into the database. Only the accounts mode creates `cubeherd.db`; single-user installs never open it.
- Backups of the dashboard: the file is one SQLite database; the doc recommends copying `cubeherd.db` together with `cubeherd.json`, and a `cubeherd db backup` command writes a consistent snapshot (`VACUUM INTO`).

4. Enforcement

One route table

Every route is registered together with the permission it needs and how the scope is derived from the request (`/n/{node}/s/{id}` gives node and server):

```
route("POST /servers/{id}/action/{action}", perm.ServerPower,  scopeServer)
route("POST /servers/{id}/files/save",      perm.FilesWrite,   scopeServer)
route("POST /servers",                      perm.ServerCreate, scopeNode)
route("GET  /settings",                     perm.SettingsManage, scopeGlobal)
```

- A test walks every handler registered on the mux and **fails if one has no permission** (or an explicit `public` marker such as `/login`). A forgotten route cannot ship open.
- Pages that mix nodes (overview, sidebar, SSE) filter per server through the same `Can` call, using one pre-computed per-request permission set so there is no query per row.
- The UI hides actions the user cannot use, but the server check is the only authority; hidden buttons are cosmetic.
- Denied access to a server the user cannot see returns **404** rather than 403, so server names do not leak.

Guard rails

- **No self-escalation:** you can only grant permissions you hold yourself, on scopes you hold them on. Editing a role is limited to permissions you have.
- **Last owner:** the last enabled Owner can be neither deleted, disabled nor demoted.
- **Instant effect:** grants are read per request (cached for seconds, invalidated on change), so removing a right applies to open sessions and running streams, which re-check on every event batch.
- **Recovery:** `cubeherd user reset-owner` on the host (or `docker exec`) creates or resets an Owner, since there is no "forgot password" mail.

Authentication details

- Local passwords keep the current hashing. Sessions become opaque random ids (hashed in the table), replacing the signed cookie that derives from the password hash. Password change, disabling, role change and "sign out everywhere" revoke sessions immediately.
- The login limiter keys on user name and IP, with the same increasing delay as today.
- CSRF protection is unchanged: every state-changing request must come from the same origin, and the session cookie is `SameSite=Strict`.

5. Audit log

The audit log is a first-class feature, written by a single `audit.Record` call next to the permission check, in the same transaction as the change where one exists.

CATEGORY	RECORDED
Authentication	Sign-in (password, SSO), failures, sign-out, session revoked, password changed, lockout
Access control	User created / disabled / deleted, role created / changed, grant added / removed / expired, SSO provider changed
Servers	Create, import, start / stop / restart, settings changed, version update, uninstall
Console	Every command sent (text recorded), player list changes
Files and world	Save, upload, delete, download of a file (path and size, **never content**), world import / export
Backups	Create, restore, delete, verify, upload
Nodes and settings	Node added / edited / removed, re-pinned, token changed (value never stored); changed setting keys with secrets redacted

Pure reads (viewing pages, streaming logs and metrics) are **not** logged, except downloads of files, world and backups, which move data out.

Properties

- **Tamper evidence:** each row stores a hash over its content and the previous row's hash. A "verify" action on the page recomputes the chain and reports the first break. This detects edits of the file; it cannot stop someone with host access who rewrites the whole chain, which is stated in the docs.
- **No edit or delete in the UI.** Only retention removes rows (default 365 days, configurable, 0 = keep forever), written as a final "pruned up to" audit entry.
- **Page:** filter by user, action, scope, outcome and time; detail drawer; CSV and JSON export (`audit.export`). Users with `audit.view` on a node or server scope see only entries for that scope.
- **Denied attempts** (authenticated but forbidden) are logged with outcome `denied`.
- **Optional forwarding** of each entry as a JSON line to stdout (`CUBEHERD_AUDIT_STDOUT=true`) so Docker log drivers can ship it to a central system.

> **Agent-side events**
> Actions that reach an agent through the dashboard are logged by the dashboard with the real user. Anything done directly on the host (docker, a shell) is outside Cubeherd's view. Later, the dashboard can pass the actor in a header so agents can keep a local log too (phase 5).

6. Single sign-on

SSO is optional and additive. It is built on **OpenID Connect** (authorization code flow with PKCE, discovery document, ID token validation). Two kinds of provider are offered in **Access → Sign-in**:

PROVIDER	CONFIGURATION
Google	A preset of the generic provider: issuer `https://accounts.google.com`, client id and secret from the Google Cloud console. Optional restriction to a Workspace domain (`hd` claim) and to verified e-mail addresses.
Generic OIDC	Issuer URL (discovery), client id and secret, scopes, button label. Works with Keycloak, Authentik, Authelia, Zitadel, Entra ID, Okta and similar.

Account linking and provisioning

- An identity is the pair (issuer, subject). First sign-in links it to an existing user by **verified e-mail** only if the admin enabled "link by e-mail"; otherwise an Owner links identities manually.
- **Auto-provisioning is off by default.** When on, new users are created with no grants (or with a configured default role) and an allowed e-mail domain list applies.
- **Role mapping (generic OIDC):** a claim (for example `groups`) can be mapped to Cubeherd groups, so grants follow the identity provider. Mapped membership is re-evaluated at each sign-in.
- Local password sign-in can be disabled once an SSO owner exists; a break-glass local Owner stays possible through the CLI.
- Client secrets are stored in the database column encrypted with a key kept in the data directory (file mode 0600), never returned by the UI.

> **Behind a reverse proxy**
> The redirect URL must be exact. A new setting *Public URL* defines it; the settings page shows the callback to register at the provider, and a "test sign-in" button checks the round trip before the setting is saved.

7. Migration, agents and limits

Turning it on

1. Settings → Access → **Enable accounts** (or `CUBEHERD_ACCOUNTS=true` for a first start).
2. The current password becomes the Owner account `admin`; its session continues. `cubeherd.db` is created.
3. Existing servers need no migration; with no further grants only Owners see them.

Turning it off again is possible while the old password hash is retained; accounts stay in the database and are ignored. The release notes call out that a downgrade to a version without accounts support falls back to the single password.

Agents

The agent token stays all-or-nothing: anybody with it can do anything on that host. RBAC protects the **dashboard**, not the agents, and users must not be given host or agent access they are not meant to have. The Nodes page text and the docs say so explicitly. Per-user tokens, read-only agent tokens or mTLS remain separate future work.

Resource impact

- Single-user installs: no change besides the larger binary.
- Accounts mode: one SQLite file, a small connection pool, an in-memory permission cache. Idle cost stays low; the audit insert is a single prepared statement.

Risks

RISK	MITIGATION
A route without a permission	Build-time test over the mux; deny by default
Privilege escalation through roles	Grant only what you hold; global-only permissions fixed by catalogue; tests for each rule
Lock-out	Last-owner rule, CLI reset, SSO break-glass
Secrets in the audit log	Central redaction list; file content and token values never recorded
SSO misconfiguration	Test sign-in before save, exact public URL, no auto-linking by default
Database growth	Retention with default 365 days, size shown on the audit page

8. Phases

Phase 1	**Foundation: SQLite, users, sessions, audit core** Store package with migrations, opt-in switch and migration of the current password, opaque revocable sessions, login per user, CSRF, audit recorder and the "authentication" events, CLI `user reset-owner`. Everything still behaves as a single Owner role.
Phase 2	**Permissions and enforcement** Permission catalogue, built-in roles, route table with the mux test, `authz.Can`, filtered overview / sidebar / SSE, 404 on invisible servers, guard rails. Grants on global, node and server scopes.
Phase 3	**Access UI** Users, groups, custom role editor with sensitive-permission warnings, grant matrix, "effective access" view for a user, sign out everywhere.
Phase 4	**Audit completion** All action events, hash chain and verify, page with filters and drawer, export, retention, stdout forwarding, scoped visibility.
Phase 5	**SSO** Generic OIDC and Google preset, linking and provisioning rules, group mapping, test sign-in, Public URL setting.
Later	**Optional** TOTP second factor, personal API tokens with a subset of the owner's rights, actor header for agent-side logs, time-limited grants in the UI, read-only agent tokens.

Verification plan
Unit tests for the permission algebra (inheritance, union, guard rails), a mux walk test, property test "no grant ⇒ no access", integration tests per role against the real handlers, an OIDC round trip against a local test provider, and the multi-host Lima lab for an end-to-end pass with users on different nodes.